编译原理课程实践报告：功亏一篑的编译器

信息科学技术学院 2200013112 林俊鑫

一、编译器概述

1.1 基本功能

本编译器基本具备如下功能：

1. 词法分析：将预处理好的高级语言代码转换为 `token` 序列，识别标识符、关键字、运算符、常量以及其他符号
2. 语法分析：将 `token` 序列转换为 `AST`，在转换过程中检查语法规范性
3. 中间代码生成：将 `AST` 转换为 `IR`，便于目标代码生成以及优化
4. 目标代码生成：将 `IR` 转换为 `RISC-V` 汇编代码，完成高级语言的编译

1.2 主要特点

本编译器的主要特点是 **模块化、可扩展性强**。

1. 模块化：编译器主要分为四个部分，每个部分分别负责四个基本功能，其中后两个部分均由若干个子模块组成
2. 可扩展性强：编译器各模块均采用面向对象编程思想，充分运用继承、多态、重载等方法，对功能增添较为友好

二、编译器设计

2.1 主要模块组成

编译器主要由四个模块组成，分别是词法分析模块、语法分析模块、中间代码生成模块、目标代码生成模块。

- 词法分析模块：
  - `sysy.l`：基于 `flex` 的分析器，将预处理好的源代码转换为 `token` 序列
- 语法分析模块：
  - `sysy.y`：基于 `bison` 的分析器，将 `token` 转换为 `AST`
- 中间代码分析模块：
  - `ast.cpp`：定义 `AST` 中各子结构，在生成 `IR` 的同时进行简单的常量检查
  - `symbol.cpp`：辅助 `IR` 生成的符号表，负责嵌套作用域、`IR` 基本块的管理
  - `operand.cpp`：原表示 `IR` 语句的操作数，在此基础上扩展为 `AST` 中各节点的综合属性，额外负责控制流状态的记录
  - `koopa.cpp`：负责 `IR` 语句的统一格式输出
- 目标代码生成模块：
  - `backend.cpp`：利用 `IR` 分析工具将 `IR` 翻译为 `RISC-V` 汇编代码，同时进行简单的类型检查

- `stack.cpp`：辅助 `RISC-V` 生成的栈，表示单个函数的栈帧
- `register.cpp`：表示 `RISC-V` 语句中的寄存器，同时也负责寄存器的分配
- `riscv.cpp`：负责 `RISC-V` 语句的统一格式输出

除了四个主要模块，编译器还包含：

- `defs.cpp`：约定模块之间信息传递格式
- `main.cpp`：控制编译器运行模式

2.2 主要数据结构

2.2.1 前端（中间代码生成）设计

2.2.1.1 ***AST***

`ast.hpp` 包含 `AST` 的数据结构的定义，并通过虚函数 `Dump` 的重载以及子类的成员函数实现 SDT，其中 `Dump` 用来进行综合属性的传递。整课 `AST` 的根为 `ProgramAST`，树中各节点定义的部分代码如下：

```
class BaseAST
{
public:
    static KoopaCode *printer;
    virtual ~BaseAST() = default;
    virtual Operand Dump() const = 0;
    friend std::ostream &operator<<(std::ostream &os, const BaseAST &ast);
};
/*
==================================================
- Program
==================================================
*/
class ProgramAST : public BaseAST
/*
==================================================
- Variable Declaration & Definition
==================================================
*/
class DeclAST : public BaseAST
class ConstDeclAST : public BaseAST
class ConstDefAST : public BaseAST
class ConstInitValAST : public BaseAST
class VarDeclAST : public BaseAST
class VarDefAST : public BaseAST
class InitValAST : public BaseAST
/*
==================================================
- Fuction Declaration & Definitino
==================================================
*/
class FuncDefAST : public BaseAST
class FuncFParamAST : public BaseAST
```

```
/*
==================================================
- Block
==================================================
*/
class BlockAST : public BaseAST
class BlockItemAST : public BaseAST
class StmtAST : public BaseAST
/*
==================================================
- Expression
==================================================
*/
class ExpAST : public BaseAST
class LValAST : public BaseAST
class PrimaryExpAST : public BaseAST
class NumberAST : public BaseAST
class UnaryExpAST : public BaseAST
class MulExpAST : public BaseAST
class AddExpAST : public BaseAST
class RelExpAST : public BaseAST
class EqExpAST : public BaseAST
class LAndExpAST : public BaseAST
class LOrExpAST : public BaseAST
class ConstExpAST : public BaseAST
```

具体以下方对应于非终结符的 `Stmt` 的 `StmtAST` 的代码为例：

- 对于非终结符的多种推导规则，我们进行子节点变量的合并，同时采用枚举类型的变量来约定具体成员变量的使用
- 对于可选的子节点，若其为空，则我们将对应成员变量赋值为空指针

```
class StmtAST : public BaseAST
{
private:
    static std::string branch_then;
    static std::string branch_else;
    static std::string branch_end;
    static std::string loop_entry;
    static std::string loop_body;
    static std::string loop_end;

public:
    enum stmt_type
    {
        LVAL,
        RETURN,
        EXP,
        BLOCK,
        IF,
        WHILE,
```

```
        BREAK,
        CONTINUE,
    } type;
    std::unique_ptr<BaseAST> lval_or_block; // 不与 stmt 合并是为了可读性
    std::unique_ptr<BaseAST> exp;
    std::unique_ptr<BaseAST> stmt;
    std::unique_ptr<BaseAST> else_stmt;
    Operand Dump() const override;
};
```

以下方对应于非终结符 `VarDef` 的 `VarDefAST` 的代码为例：

- 对于不确定数量的同类子节点，采用双端队列进行存储
- 对于继承属性 `BType`，采用静态变量方式进行传递

```
class VarDefAST : public BaseAST
{
public:
    static BType base_type;
    enum var_def_type
    {
        IDENT,
        INITVAL,
    } type;
    std::string ident;
    std::deque<std::unique_ptr<BaseAST>> array_sizes;
    std::unique_ptr<BaseAST> init_val;
    Operand Dump() const override;
};
```

2.2.1.2 ***Symbol & SymbolTables***

`symbol.hpp` 包含 `Symbol` 与 `SymbolTables` 的定义，其中 `Symbol` 负责记录有关属性和值，代码如下：

```
class Symbol
{
public:
    enum symbol_type
    {
        CONST,
        VAR,
        CONST_ARRAY,
        VAR_ARRAY,
        FUNC,
    } type;
    int val;
    BType ret_or_array;
    Symbol();
    Symbol(const symbol_type &t, const int &v = 0);
    Symbol(const symbol_type &t, const BType &data_type);
    Symbol(const Symbol &s);
```

```
    BType FuncRetType();
    BType ArrayType();
    Symbol &SetVal(const int &v);
};
```

`SymbolTables` 负责符号表的管理。我们约定一个作用域对应一个符号表，并使用单例模式 `symbol_tables` 管理一个程序的所有作用域，其中 `vert_symtabs` 用来管理嵌套作用域（纵向管理），`sub_symtab_sum` 用来管理兄弟作用域（横向管理）。后续由于循环和条件分支的加入，并且二者的本质都是跳转指令，因此我们采用统一的变量 `branch_count` 来分配基本块标签，用变量 `loop_marks` 记录嵌套循环对应的标签。代码如下：

```
class SymbolTables
{
private:
    static SymbolTables *pSymbolTables;
    typedef std::unordered_map<std::string, Symbol> symbol_table_t;
    std::vector<symbol_table_t> vert_symtabs; // 嵌套作用域
    std::vector<int> sub_symtab_num;          // 兄弟作用域
    std::stack<int> loop_marks;               // 嵌套循环
    int branch_count;                         // 基本块标签计数
    SymbolTables() = default;
    ~SymbolTables() = delete;
    SymbolTables(const SymbolTables &) = delete;
    SymbolTables &operator=(const SymbolTables &) = delete;

public:
    static SymbolTables *GetSymbolTables();
    void NewSymbolTable();
    void DeleteSymbolTable();
    std::string Mark(const std::string &name, const int &mark);
    void RecordSymbol(const std::string &ident, const Symbol &value);
    std::string GetName(const std::string &ident);
    Symbol GetSymbol(const std::string &ident);
    int NewBranchMark();
    void PushLoop(const int &loop_mark);
    int GetTopLoop();
    void PopLoop();
};

extern SymbolTables *symbol_tables;
```

2.2.1.3 ***Operand***

在设计初期，`operand.hpp` 简单地定义了 `IR` 操作数 `Operand`；后来由于控制流的加入，节点需要子节点更多的信息，因此考虑将 `Operand` 扩展，成为附带控制流信息的节点的综合属性。

```
class Operand
{
private:
    union operand
    {
        int reg_no;
```

```
        int imm_value;
    } value;
    static int reg_count;
public:
    enum operand_type
    {
        REG,
        IMM,
    } type;
    Operand();
    Operand(const Operand &operand);
    Operand(const operand_type &t, const int &v = 0);
    ~Operand();
    int ImmValue() const;
    bool IsReg() const;
    bool IsNormal() const;            // 扩展
    Operand &SetAsReturnMark();       // 扩展
    bool IsReturnMark();              // 扩展
    Operand &SetAsLoopInterruption(); // 扩展
    bool IsLoopInterruption();        // 扩展
    Operand operator !=(const int &v);
    friend std::ostream &operator<<(std::ostream &os, const Operand &operand);
};
```

2.2.2 后端（目标代码生成）设计

2.2.2.1 ***Backend***

`backend.hpp` 定义的类 `Backend` 实质上只是个管理重载的目标代码生成函数 `Visit` 的类。对于有返回值的 `IR` 语句，我们会额外将其基类型下的值 `value` 传入 `Visit` 便于后续进行查找。代码如下：

```
class Backend
{
public:
    static bool pass_entry;
    Backend(const char *koopa_str);
    ~Backend();
    friend std::ostream &operator<<(std::ostream &os, Backend &backend);
    void Visit(std::ostream &os);

private:
    koopa_raw_program_t raw_program;
    koopa_raw_program_builder_t builder;
    Stack stack;
    RiscvCode printer;
    int GetAllocSize(const koopa_raw_type_t &ty);
    int IsPowOfTwo(const int &number);
    void Visit(const koopa_raw_slice_t &slice);
    void Visit(const koopa_raw_function_t &func);
    void Visit(const koopa_raw_basic_block_t &bb);
    void Visit(const koopa_raw_return_t &ret);
    void Visit(const koopa_raw_value_t &value);
```

```
    void Visit(const koopa_raw_store_t &store);
    void Visit(const koopa_raw_branch_t &branch);
    void Visit(const koopa_raw_jump_t &jump);
    void Visit(const koopa_raw_aggregate_t &aggregate);
    void Alloc(const koopa_raw_value_t &alloc);
    void Visit(const koopa_raw_value_t &value, const koopa_raw_binary_t &binary);
    void Visit(const koopa_raw_value_t &value, const koopa_raw_load_t &load);
    void Visit(const koopa_raw_value_t &value, const koopa_raw_call_t &call);
    void Visit(const koopa_raw_value_t &value, const koopa_raw_global_alloc_t
&global_alloc);
    void Visit(const koopa_raw_value_t &value, const koopa_raw_get_ptr_t &get_ptr);
    void Visit(const koopa_raw_value_t &value, const koopa_raw_get_elem_ptr_t
&get_elem_ptr);
};
```

2.2.2.2 ***Stack***

`stack.hpp` 定义了类 `Stack` 用来模拟单个函数的栈帧，与传统栈帧不同的是，`Stack` 的参数构造区在栈帧的最低地址，局部变量构造区在参数构造区的高地址方向，以此便于参数构造和局部变量存储时的占用区域大小检查。`local_off_map` 与 `temp_off_map` 分别记录了存储在栈上的变量和构造的参数在栈上的偏移量。代码如下：

```
class Stack
{
private:
    int total_capacity;
    int ra_capacity;
    int local_capacity;
    int temp_capacity;
    int local_offset;
    int temp_offset;
    typedef std::unordered_map<koopa_raw_value_t, int> offset_map_t;
    offset_map_t local_off_map;
    offset_map_t temp_off_map;

public:
    Stack();
    Stack(const int &total_cap, const int &temp_cap, const int &ra_cap);
    bool Find(const koopa_raw_value_t &value);
    void Push(const koopa_raw_value_t &value, const int &type_size, const bool &temp =
false);
    void Pop(const int &type_size, const bool &temp = false);
    int GetOffset(const koopa_raw_value_t &value, const bool &temp = false);
    int GetStackSize();
    bool SaveRa();
    int RaOffset();
};
```

2.2.2.3 ***Register***

设计初期，`register.hpp` 定义了 `RISC-V` 语句的操作数 `Register`，即寄存器；后来由于数组初始化对于单纯的SSA提出了挑战，需要考虑寄存器的分配，因此 `Register` 也在调用构造函数时扩展了寄存器分配的过程。代码如下：

```
class Register
{
private:
    static koopa_raw_value_t *reg_record;
    static std::unordered_map<koopa_raw_value_t, int> allocated_value;  // 扩展
    static int usable_reg_num;
    static int ret_reg_no;
    static int sp_no;
    static int ra_no;
    static int curr_reg;                                                // 扩展
    int reg_no;
    void GetReg();                                                      // 扩展
    void Unallocate();

public:
    enum special_register_type
    {
        ZERO,
        RET,
        SP,
        RA,
        PARAM,
        IMM,
    };
    static int PARAM_REG_NUM;
    static void Initialize();                                           // 扩展
    static bool Cached(const koopa_raw_value_t &value);                 // 扩展
    Register(const special_register_type &type = ZERO, const int &id = 0);
    Register(const koopa_raw_value_t &value, bool *cached = nullptr);   //  （增加功能）扩展
    ~Register();
    friend std::ostream &operator<<(std::ostream &os, const Register &reg);
};
```

2.2.3 其他设计

2.2.3.1 ***BType***

由于数组类型的加入，数据类型的表示与传递变得复杂，并且后端的 `Backend` 与统一格式输出 `RiscvCode` 均需要使用数据类型的信息，因此 `defs.hpp` 中实现了数据类型类 `BType`，用以规定合法数据类型并提供相应的输出表示。其定义如下：

```
class BType
{
public:
    enum data_type
    {
        VOID,
```

```
        INT,
        ARRAY,
    } type;
    BType *array_base_type;
    int array_size;
    BType();
    BType(const data_type &t, std::deque<Operand> array_sizes = {});
    BType(const BType &t);
    ~BType();
    BType &operator=(const BType &t);
    bool IsVoid() const;
    bool IsInt() const;
    bool IsArray() const;
    std::deque<int> ArraySizes() const;
    bool operator==(const BType &t) const;
    friend std::ostream &operator<<(std::ostream &os, const BType &type);
};
```

2.3 主要设计考虑及算法选择

2.3.1 符号表的设计考虑

我们约定一个作用域用一个符号表进行管理，对于多个作用域，

- 从嵌套作用域的角度，以向量形式按照作用域的深度存储当前及其所有祖父作用域。在实现上，我们约定程序本身拥有一个作用域，即第0层符号表，全局变量与函数的符号存储在第0层符号表，函数内局部变量从第1层及以下符号表存起，从而实现同名变量的覆盖
- 从兄弟作用域的角度，以向量形式存储不同深度作用域的子作用域个数，通过为变量添加兄弟作用域的计数编号来避免同名变量的跨域使用

2.3.2 寄存器分配策略

每次为变量分配寄存器时，先查看寄存器分配表中该变量是否已分配，

- 若已分配，则使用分配的寄存器
- 若未分配，则将计数器对应的寄存器解除分配关系，然后将该寄存器分配给当前变量，计数器数值加一

对于每个函数，计数器初始值对应的寄存器均为 t0，这样在将函数参数转存栈上时不会造成参数被覆盖。

该寄存器分配策略是基于循环的on-the-fly的策略，对于数组操作的空间局部性很差，对于循环的时间局部性也很差。本人还尝试了其他的一些on-the-fly策略，但是都没有成功。由于时间原因，对生成的目标代码的寄存器染色分配也没实现。

2.3.3 采用的优化策略

前端使用的优化：

- 短路求值：主要用于 `LOrExp` 和 `LAndExp` 中
- 常量传播：若表达式的操作数均为常量，则表达式的结果也为常量
- 消除死代码：若循环或条件分支的条件为常量，则直接跳转到目标基本块，不进行死代码的翻译；若跳转分支的两个结果都包含 `ret` 则无需再进行后续 `IR` 生成

2.3.4 其它补充设计考虑

- 不完全遵循 SSA 的要求，取消 `load` 压栈操作：
  - 原因：在后端进行目标代码生成时，对于 `IR` 中的 `load` 指令，按照 SSA 的要求，需要将结果存储在栈上，导致后续操作额外多一次读取操作，总计多一次读和写操作，会造成性能极大降低
  - 要求：每次从栈上取变量前需要检查是否已分配寄存器，若已分配则直接使用，否则再执行一次 `load`
  - 合理性：`load` 指令之前一定有 `Store` 指令，因此不需要再次进行存储；若 `load` 指令的结果在寄存器被刷新前使用，则总计减少了一次读和写的开销，若在被刷新后使用，则总计减少了一次写的开销，两者都比原先压栈操作更好
  - 局限性：需要好的寄存器分配策略（显然我目前使用的分配策略效果不佳）
- 探索好的on-the-fly寄存器分配策略（未成功/暂时未实现）
  - 栈式寄存器分配：
    - 方法：每次 `getelemptr` 操作存放的寄存器都保留不被清除，等该指针使用完再释放
    - 缺点：难以实现，尤其是数组初始化的时候，无法确认什么时候指针不会再被使用
  - 基于深度参数的寄存器分配：
    - 方法：考虑附带数组维数参数的寄存器分配，通过参数指定哪个寄存器可以在下一次操作中重新分配
    - 缺点：无法解决数组维数超过寄存器总个数的情况

三、编译器实现

3.1 各阶段编码细节

（事实上，在进行未通过测试的Lv的代码编写时，本人经常对于已完成Lv的功能的代码部分进行修改，这里选择将这部分修改放在它原来应该在的位置进行说明。）

Lv0. 环境配置

在 `Makefile` 中新添头文件路径并默认关闭DEBUG：

```
DEBUG ?= 0
INC_DIRS += $(TOP_DIR)/include
```

在 bash 中设置别名便于本地测试：

```
alias test_koopa='./build/compiler -koopa debug/hello.c -o hello.koopa'
alias test_riscv='./build/compiler -riscv debug/hello.c -o hello.riscv'
```

Lv1. main函数和Lv2. 初试目标代码生成

- 前端 & 后端：
  - `IR` 和 `RISC-V` 的语句输出各自用一个类来统一执行

Lv3. 表达式

- 前端 & 后端：
  - `IR` 和 `RISC-V` 的语句操作数各自用一个类来抽象，重载操作符<<便于输出语句。 `Number` 虽然是语法分析时的常量，但是仍然可以用一个 `AST` 节点包装，这样简化了后续 `IR` 翻译时的处理逻辑

Lv4. 常量和变量

- 前端：
  - `IR` 操作数可以扩展为 `AST` 节点的综合属性，因为变量类型的 `Symbol` 的数值域并没有被使用，所以可以用来干一些别的事，比如作为填-1作为表示返回信号，并递归传递回函数节点，以此来判断一个函数的 `IR` 生成结束时是否已经生成 `ret` 指令
  - 变量的类型可以作为继承属性传递给 `VarDef`（以及后续Lv9的 `ConstVarDef`）

Lv5. 语句块和作用域

- 前端：
  - 将程序视为第0层符号表，每层符号表变量的变量名添加层数标签，实现了全局变量的管理，简化了函数名与变量名的判定。引入子符号表的计数，使得子符号表可以通过父符号表获取兄弟符号表的信息，实现了变量的跨域隔离，避免了同名变量的多次声明

Lv6. if语句

- 语法分析：将 `Stmt` 分为 `MatchedStmt` 和 `UnmatchedStmt`，从而实现悬空的解决
- 前端：
  - 如果条件是常量则可以消除死代码
  - `LAndExp` 和 `LOrExp` 可以进行短路求值
  - 当且仅当真子节点和假子节点的综合属性都是返回信号时当前节点的综合属性为返回信号，且此时不需要再进行函数后续 `IR` 生成

Lv7. while语句

- 前端：
  - 对于条件分支，当且仅当真子节点和假子节点的综合属性均为控制流中断信号时当前节点的综合属性为控制流中断信号，当且仅当真子节点和假子节点的综合属性都是返回信号时当前节点的综合属性为返回信号

Lv8. 函数和全局变量

- 前端：
  - 函数参数先转存为局部变量，这样在后端翻译时 `load` 就不用考虑这个区别
  - 由于参数转存发生在进入基本块之前，因此是否输出 `entry` 标签和创建新符号表是 `Block` 的继承属性
  - 由于全局变量声明和局部变量不一样，因此是否为全局状态是 `Decl` 的继承属性
- 后端：
  - 过量参数存在栈上，参数构造区如果在栈帧底部会比较方便传递参数
  - 全局变量使用 `la` ，需要进行类型判断

Lv9. 数组

- 前端：
  - 指针数组只会出现在数组参数中，其余指针要么为全局数组要么为局部数组，均为数组指针，因此可以将两种类型用 `BType` 统一，区别在于第0维的长度，进而可以任意两种数据类型的比较。在输出类型表示时若为0则输出前缀*，否则递归用中括号进行输出
  - 深度优先的数组初始化的空间局部性较好，便于后端优化
- 后端：
  - 指针数组和数组指针的步长不一致，其余相似
  - `load` 参数时如果是指针，则需要解引用

3.2 工具软件介绍（若未使用特殊软件或库，则本部分可略过）

1. `flex/bison`：用于词法和语法自动分析，其中 `flex` 用正则表达式解析词法得到 `token` 流，`bison` 根据文法生成语法分析器构造 `AST`
2. `libkoopa`：一款全自动 `Koopa IR` 自动处理库，部分类型的属性在名字上意思很接近，需要仔细品味体会才能找到区别
3. `make`：自动编译工具，通过文件修改日期检查的方式进行大型项目的增量编译更新
4. `gdb-multiarch`：一款能运行在 `qemu` 上的调试器，可以直接查看栈帧与内存，对于后端调试效果显著

四、实习总结

4.1 收获和体会

第一次字面意义上没有重构地写完了一个上千行的项目，体验到了课程实验设置的精妙和教学组的用心。实验初期从零到一的过程摸索了很久，事后发现需要多对照课堂教学内容才能理解其中关键，因此实验与课堂学习缺一不可。当然最后还是很遗憾没能拿满 Lv9 的 `RISC-V` 部分的分数。

4.2 学习过程中的难点，以及对实习过程和内容的建议

`libkoopa` 库中对于各个属性的详解（在写代码过程中写混了很多次）。

4.3 对老师讲解内容与方式的建议

暂无。